AYUDANTÍA 02 · FAGOB 2026

Procesamiento de datos con tidyverse

Métodos Cuantitativos para la Administración Pública

Cristóbal Mejías · Facultad de Gobierno Viernes 27 de marzo, 2026

Agenda de hoy

¿Qué haremos?

1. Repaso y carga de datos
2. Tipos de variables en R
3. `select()` + `rename()`
4. `filter()`
5. `mutate()` + `recode()`
6. Valores perdidos (NA)
7. Transformar variables
8. Guardar la base procesada
9. El pipe `%>%`

Duración: 90 minutos

Al final de la sesión: Tendrás una base de datos limpia, procesada y guardada — lista para analizar en la próxima ayudantía.

00

Punto de partida

¿Qué hicimos en AY-1?

- Creamos un **proyecto en R** con estructura IPO
- Aprendimos a cargar datos con `read_csv()`
- Exploramos la CASEN con `dim()`, `names()`, `summary()`

Problema: los datos que tenemos aún no están listos para analizar. Las variables tienen **códigos numéricos** en vez de etiquetas, hay **valores especiales** (-88, -99) y los tipos de datos no son los correctos.

Hoy aprendemos a limpiar y procesar.

Recordatorio: sistema IPO

```
📂 ayudantia-2/
├── input/         ← datos originales (NO se modifican)
├── procesamiento/ ← script de hoy
└── output/        ← casen procesada lista para analizar
```

Principio: el script debe ser **reproducible**. Cualquier persona que lo corra desde cero debe obtener exactamente los mismos resultados.

Paquetes de hoy

```r
library(pacman)
pacman::p_load(
  tidyverse,    # colección de paquetes para manipulación de datos
  dplyr,        # para manipular datos (select, filter, mutate ... )
  car,          # para recodificar variables con recode()
  magrittr,     # para el operador pipe %>%
  sjmisc,       # para explorar y describir variables (frq)
  sjPlot,       # para visualizar la base completa (view_df)
  summarytools  # para resumen detallado de la base (dfSummary)
)

options(scipen = 999) # desactivar notación científica
rm(list = ls())       # limpiar ambiente de trabajo
```

Cargar la base de datos

Como hoy no traen la base descargada, la cargamos **desde GitHub**:

```
casen <- haven::read_dta(
  "https://raw.githubusercontent.com/Cristobal-Mejias-G/Clase-2-MC-FAGOB/main/input/casen_fagob.dta"
)
```

Esta base tiene **5.000 casos** y **16 variables** de la CASEN 2024. Al cargar desde `.dta` las variables llegan como `haven_labelled`: números con etiquetas pegadas, igual que en el archivo original de la encuesta. Nuestro trabajo de hoy es convertirlos al tipo correcto.

Primera mirada: ver toda la base

```r
# Resumen completo: distribuciones, NAs, tipos de dato
view(dfSummary(casen, headings = FALSE, method = "render"))

# Vista de variables con etiquetas y distribución visual
view_df(casen, max.len = 100)
```

`dfSummary()` abre una tabla interactiva en el Viewer de RStudio con estadísticos, histogramas y porcentaje de NAs para cada variable.

`view_df()` muestra las etiquetas de la base (cuando las tiene) junto con la distribución de cada variable.

Primera mirada: variables específicas

```r
# Tabla de frecuencias detallada para una variable categórica
sjmisc::frq(casen$region)
sjmisc::frq(casen$sexo)

# Estadísticos de una variable numérica
mean(casen$o10, na.rm = TRUE)  # promedio horas trabajadas
summary(casen$o10)             # mín, Q1, mediana, media, Q3, máx
```

¿Cuándo usar cada uno?

- Variables con categorías → `frq()` muestra frecuencias y porcentajes
- Variables numéricas → `summary()` muestra estadísticos claves

01

Tipos de variables en R

¿Por qué importa el tipo de variable?

R necesita saber qué tipo de dato tiene una variable para:

- Hacer las **operaciones correctas** *(no puedes calcular el promedio de texto)*
- Aplicar las **funciones adecuadas**
- **Graficar** de forma apropiada

```
# Explorar tipos
class(casen$sexo)
class(casen$edad)
class(casen$o10)
```

Los tipos que encontraremos

Tipo en R	¿Qué es?	Ejemplo
`numeric` / `double`	Número con decimales	`edad`, `o10`, `yoprcor`
`integer`	Número entero	Menos común en CASEN
`character`	Texto	Variables abiertas
`factor`	Categorías con niveles	Nuestro destino para variables nominales/ordinales
`haven_labelled`	**Tipo especial de Stata/SPSS**	**La mayoría de variables CASEN**

¿Qué es `haven_labelled`?

Cuando haces `class(casen$sexo)` probablemente ves:

[1] "haven_labelled" "vctrs_vctr" "double"

Léelo de abajo hacia arriba:

- `double` → en el fondo es un número (1, 2, 3...)
- `vctrs_vctr` → sigue las convenciones del tidyverse (detalle técnico)
- `haven_labelled` → tiene **etiquetas pegadas**: el `1` sabe que significa "Hombre" y el `2` que significa "Mujer"

> Es el formato que usa Stata. Los datos son **números** pero llevan las etiquetas como mochila. Por eso `view_df()` puede mostrarte los nombres aunque internamente sean códigos.

La regla práctica (actualizada)

Antes de analizar cualquier variable, haz `class()` y decide qué hacer:

Lo que ves en `class()`	Qué significa	Qué hacer
`numeric` / `double`	Número puro	Listo si tiene sentido matemático
`character`	Texto	Convertir a `factor` si es categórica
`factor`	Categorías	Ya está bien
`haven_labelled`	Número con etiqueta	**Recodificar y convertir a `factor`**

En la CASEN casi todo llega como `haven_labelled`. Nuestro trabajo hoy es convertir cada variable al tipo correcto según su nivel de medición.

02

`select()` + `rename()`

¿Qué hace `select()`?

`select()` nos permite **elegir columnas** y, opcionalmente, **renombrarlas**.

La CASEN tiene nombres crípticos (`yoprcor`, `o10`, `e3`...). Aprovechamos `select()` para renombrarlos a algo entendible:

nuevo_nombre = nombre_original

Seleccionar y renombrar las 16 variables

```r
casen_proc <- select(casen,
  # Identificación
  region,
  zona            = area,
  # Demografía
  sexo,
  edad,
  estado_civil    = ecivil,
  # Educación
  nivel_educ      = educc,
  asiste_educ     = e3,
  # Trabajo
  actividad       = activ,
  categ_ocup      = o15,
  horas_trab      = o10,
  contrato        = o19,
  # Ingresos
  ingreso         = yoprcor,
  ingreso_pc      = ypchtotcor,
  # Pobreza y características
  pobreza         = pobreza_severa,
  pueblo_orig     = pueblos_indigenas,
  lugar_nac
)
```

Convertir `haven_labelled` a numeric

Antes de recodificar, eliminamos el tipo `haven_labelled`. Ya exploramos las etiquetas con `view_df()` y `frq()` — ahora necesitamos números limpios:

```
# Convertir todas las variables a numeric puro
casen_proc <- mutate(casen_proc, across(everything(), haven::zap_labels))

# Verificar: ahora todo es numeric
glimpse(casen_proc)
```

`zap_labels()` elimina las etiquetas Stata/SPSS pero conserva los valores numéricos intactos. Los `-88` y `-99` siguen ahí — los trataremos en la sección de NAs.

03

`filter()`: filtrar filas

¿Qué hace `filter()`?

`filter()` nos permite **elegir filas** que cumplen una condición lógica.

Operador	Significado	Ejemplo
`==`	igual a	`sexo == 1`
`!=`	distinto de	`actividad != 3`
`>` / `<`	mayor / menor	`edad > 18`
`>=` / `<=`	mayor o igual / menor o igual	`edad >= 60`
`&`	Y (ambas condiciones)	`sexo == 2 & edad > 18`
`%in%`	pertenece a un conjunto	`region %in% c(13, 5, 8)`

`filter()` en práctica

```r
# Solo personas mayores de 18
adultos <- filter(casen_proc, edad > 18)

# Solo personas ocupadas (actividad == 1)
ocupados <- filter(casen_proc, actividad == 1)

# Región Metropolitana (13) y Valparaíso (5)
rm_valpo <- filter(casen_proc, region %in% c(13, 5))

# Verificar siempre
nrow(casen_proc)
nrow(adultos)
```

Si `nrow()` no cambia, probablemente la condición está mal escrita. Revisa el tipo de la variable con `class()` antes de filtrar.

04

`mutate()` + `recode()`

El flujo: `mutate()` + `recode()`

Para cambiar los **valores** de una variable usamos dos funciones juntas:

- `mutate()` → **crea o modifica** una columna
- `recode()` → **cambia los valores** dentro de esa columna

Sintaxis de `car::recode()`: los pares código-etiqueta van dentro de un string separados por `;`. Las etiquetas nuevas van entre comillas simples `'`.

Recodificar `sexo`

```r
casen_proc <- mutate(casen_proc,
                     sexo = car::recode(sexo,
                                        recodes = c("1 = 'Hombre'; 2 = 'Mujer'"),
                                        as.factor = TRUE,
                                        levels = c("Hombre", "Mujer")))

# Verificar
table(casen_proc$sexo)
class(casen_proc$sexo)
```

Recodificar `actividad`

```
casen_proc <- mutate(casen_proc,
                     actividad = car::recode(actividad,
                                             recodes = c("1 = 'Ocupado/a';
                                                          2 = 'Desocupado/a';
                                                          3 = 'Inactivo/a'"),
                                             as.factor = TRUE,
                                             levels = c("Ocupado/a",
                                                        "Desocupado/a",
                                                        "Inactivo/a")))
table(casen_proc$actividad)
```

Recodificar `pobreza`

```r
casen_proc <- mutate(casen_proc,
                     pobreza = car::recode(pobreza,
                                           recodes = c("1 = 'Pobreza severa';
                                                       2 = 'Pobreza por ingresos';
                                                       3 = 'Pobreza multidimensional';
                                                       4 = 'No pobre'"),
                                           as.factor = TRUE,
                                           levels = c("Pobreza severa",
                                                      "Pobreza por ingresos",
                                                      "Pobreza multidimensional",
                                                      "No pobre")))
table(casen_proc$pobreza)
```

`levels =` controla el orden de aparición en tablas y gráficos. El primer nivel es la categoría de referencia en regresiones.

Verificar el estado actual

```r
# ¿Los tipos quedaron bien?
glimpse(casen_proc)

# ¿Los valores son los esperados?
sjmisc::frq(casen_proc$sexo)
sjmisc::frq(casen_proc$actividad)
sjmisc::frq(casen_proc$pobreza)
```

05

Valores perdidos (NA)

-88, -99 y NA: son cosas distintas

En la CASEN hay tres tipos de "dato que falta":

Valor	Significado en la encuesta	En R
`-88`	No aplica (ej. horas trabajadas de un inactivo)	Número negativo, **no es NA**
`-99`	No sabe / No responde	Número negativo, **no es NA**
`NA`	Dato genuinamente ausente en R	`NA` (valor especial de R)

Problema: R no sabe que -88 y -99 son "ausentes". Los trata como números reales. Si calculas el promedio de `horas_trab` sin corregir esto, el resultado será incorrecto.

Paso 1: identificar los valores especiales

```r
# ¿Cuántos -88 y -99 hay en horas_trab?
table(casen_proc$horas_trab)

# ¿Y en contrato?
table(casen_proc$contrato)

# Ver todos los NAs reales actuales
colSums(is.na(casen_proc))
```

Paso 2: convertir -88 y -99 a NA

```r
casen_proc <- mutate(casen_proc,
  ingreso    = car::recode(ingreso,    recodes = c("-88 = NA; -99 = NA")),
  ingreso_pc = car::recode(ingreso_pc, recodes = c("-88 = NA; -99 = NA")),
  horas_trab = car::recode(horas_trab, recodes = c("-88 = NA; -99 = NA")),
  contrato   = car::recode(contrato,   recodes = c("-88 = NA; -99 = NA")),
  categ_ocup = car::recode(categ_ocup, recodes = c("-88 = NA; -99 = NA"))
)

colSums(is.na(casen_proc))
colMeans(is.na(casen_proc))
```

Usamos `car::recode()` variable por variable para ser explícitos sobre cuáles tienen valores especiales. Solo las variables donde -88/-99 tienen sentido como "dato ausente".

Paso 3: dos objetos — con y sin NA

```r
# Objeto con NAs (lo que tenemos ahora)
# casen_proc ← ya lo tenemos

# Objeto sin NAs
casen_proc_na ← na.omit(casen_proc)

# ¿Cuántos casos perdemos?
nrow(casen_proc)
nrow(casen_proc_na)
```

NAs en operaciones estadísticas

Sin `na.rm`, el resultado es `NA` en silencio:

```
mean(casen_proc$ingreso)    # NA ← no da error, pero tampoco resultado
```

Con `na.rm = TRUE`:

```
mean(casen_proc$ingreso, na.rm = TRUE)       # promedio correcto
median(casen_proc$horas_trab, na.rm = TRUE)  # mediana correcta
```

Regla: casi todas las funciones estadísticas de R tienen `na.rm`. Úsalo siempre que tu variable tenga NAs.

06

Transformar variables

Crear nuevas categorías

A veces necesitamos crear categorías **a partir de variables numéricas**:

- `edad` → tramos etarios
- `ingreso` → niveles de ingreso

Función	Cuándo usarla
`if_else()`	Condición binaria (dos salidas)
`case_when()`	Múltiples condiciones

`if_else()` : dos resultados

```
casen_proc_na <- mutate(casen_proc_na,
  adulto_mayor = if_else(edad >= 60, "Adulto mayor", "No adulto mayor"),
  adulto_mayor = factor(adulto_mayor,
                        levels = c("No adulto mayor", "Adulto mayor"))
)

table(casen_proc_na$adulto_mayor)
```

`case_when()` : múltiples condiciones

```
casen_proc_na <- mutate(casen_proc_na,
  tramo_edad = case_when(
    edad < 18              ~ "Menor de edad",
    edad >= 18 & edad < 40 ~ "Adulto joven",
    edad >= 40 & edad < 60 ~ "Adulto",
    edad >= 60             ~ "Adulto mayor"
  ),
  tramo_edad = factor(tramo_edad,
                      levels = c("Menor de edad", "Adulto joven",
                                 "Adulto", "Adulto mayor"))
)

table(casen_proc_na$tramo_edad)
```

`case_when()` con ingresos

```
casen_proc_na <- mutate(casen_proc_na,
  tramo_ingreso = case_when(
    ingreso < 300000                      ~ "Bajo",
    ingreso >= 300000 & ingreso < 700000 ~ "Medio",
    ingreso >= 700000                     ~ "Alto"
  ),
  tramo_ingreso = factor(tramo_ingreso,
                         levels  = c("Bajo", "Medio", "Alto"),
                         ordered = TRUE)
)

table(casen_proc_na$tramo_ingreso)
```

07

Guardar

Guardar la base procesada

```r
# Guardar en formato RDS (formato nativo de R, más eficiente)
saveRDS(casen_proc_na, file = "output/casen_proc.rds")

# Para cargar en la próxima ayudantía:
# casen_proc <- readRDS("output/casen_proc.rds")
```

¿Por qué RDS y no CSV?

- Conserva los tipos de datos: `factor`, `ordered`, etc.
- Si guardas como `.csv` y vuelves a cargar, los `factor` se convierten a `character` y hay que rehacer el procesamiento.
- `.rds` guarda el objeto R exactamente como está.

08

Práctica autónoma

Tu turno (10 min)

Con `casen_proc` (el objeto **con NAs**), trabaja con las variables **`zona`**, **`estado_civil`** y **`nivel_educ`**.

Instrucciones:

1. Explora con `sjmisc::frq()` y `class()` — ¿qué tipo tienen? ¿qué valores?
2. Recodifica `zona` (1=Urbano, 2=Rural) con `recode()` y conviértela a `factor`
3. Recodifica `nivel_educ` (0=Sin educ. → 6=Superior completa) con `car::recode()` y conviértela a `factor` con `levels` en orden ascendente
4. Filtra solo personas de zona urbana
5. Calcula el promedio de `ingreso_pc` por zona con `mean( ... , na.rm = TRUE)`

> Consulta el libro de códigos si no recuerdas qué significa cada código.

09

El pipe `%>%`

¿Qué es el pipe?

El operador `%>%` significa: **"toma esto y pásalo a la siguiente función"**.

```
x %>% f()           # equivale a f(x)
x %>% f() %>% g()   # equivale a g(f(x))
```

Atajo de teclado: `Ctrl + Shift + M` (Windows) | `Cmd + Shift + M` (Mac)

Sin pipe vs. con pipe

Sin pipe *(lo que hemos practicado hoy)*:

```
# Quitar haven_labelled: convertir a numeric puro para poder recodificar
casen <- mutate(casen, across(everything(), haven::zap_labels))

casen_proc2 <- select(casen, sexo, edad, actividad = activ,
                      ingreso = yoprcor, pobreza = pobreza_severa)
casen_proc2 <- filter(casen_proc2, edad >= 18)
casen_proc2 <- mutate(casen_proc2,
                      sexo = car::recode(sexo,
                                         recodes = c("1 = 'Hombre'; 2 = 'Mujer'"),
                                         as.factor = TRUE,
                                         levels = c("Hombre", "Mujer")))
```

Con pipe *(equivalente)*:

```
casen_proc2 <- casen %>%
  select(sexo, edad, actividad = activ,
         ingreso = yoprcor, pobreza = pobreza_severa) %>%
  filter(edad >= 18) %>%
  mutate(sexo = car::recode(sexo,
                            recodes = c("1 = 'Hombre'; 2 = 'Mujer'"),
```

Resumen de la sesión

Paso	Función(es)	¿Para qué?
Explorar	`dfSummary()`, `view_df()`, `frq()`	Ver la base antes de tocarla
Seleccionar	`select()`, `rename()`	Elegir y renombrar variables
Filtrar	`filter()`, operadores lógicos	Quedarse con ciertos casos
Recodificar	`mutate()`, `recode()`, `factor()`	Convertir códigos en etiquetas
NAs	`na_if()`, `drop_na()`, `na.rm`	Tratar valores especiales y faltantes
Transformar	`if_else()`, `case_when()`	Crear nuevas categorías
Guardar	`saveRDS()`	Preservar el trabajo para AY-3
Encadenar	`%>%`	Escribir código más limpio

¡Nos vemos en la siguiente sesión!

AY-3 · 10 de abril Análisis descriptivo, bivariado y gráficos con `ggplot2`

Cristóbal Mejías · Facultad de Gobierno